Date of performance: 20/03/2023
Date of submission: 10/04/2023

Experiment No.: 3

Name: Vijay J. Harkare
SAP Id.: 60004200079
Subject: Ubiquitous Computing (Honours)
Division: A
Batch: H2
Branch: Computer Engineering

Aim: To implement context-aware system.

Theory:
Context awareness refers, in information and communication technologies, to a capability to take into account the situation of entities, which may be users or devices, but are not limited to those. Location is only the most obvious element of this situation. Narrowly defined for mobile devices, context awareness does thus generalize location awareness. whereas location may determine how certain processes around a contributing device operate, context may be applied more flexibly with mobile users, especially with users of smart phones. Context awareness originated as a term from ubiquitous computing or as so-called pervasive computing which sought to deal with linking changes in the environment with computer systems, which are otherwise static. The term has also been applied to business theory in relation to contextual application design and business process management issues.

Mainly, in computer science, context awareness refers to the idea that computers can both sense, and react based on their environment. Devices may have information about the circumstances under which they are able to operate and based on rules, or an intelligent stimulus, react accordingly.
Context-aware systems are concerned with the acquisition of context (e.g. using sensors to perceive a situation), the abstraction and understanding of context (e.g. matching a perceived sensory stimulus to a context), and application behaviour based on the recognized context (e.g. triggering actions based on context). As the user's activity and location are crucial for many applications, context awareness research mainly focuses on location awareness and activity recognition.

Conclusion:

In this experiment, we studied context awareness, its types, and how to build context aware systems. We also built a system which takes in natural language input from user, analyzes it, processes it, then classifies it based on pre-defined rules, and based on the understood context, related locations are visually mapped on a map and displayed to the user. Thus, we have implemented a location-aware system (since current location of user is also considered). Thus, based on our observations, context-aware systems make a system dynamic and responsive to external environment, and thus enables truly pervasive computing.

Sundaram®
FOR EDUCATIONAL USE

Code:

1. Code to classify the context sentences, using Universal Sentence Encoder and similarity matching:

```
import pandas as pd

import tensorflow_hub as hub

import numpy as np

from sklearn.metrics.pairwise import cosine_similarity


# define the categories list

categories = ['restaurant', 'ice cream', 'cafe', 'college', 'school', 'playground', 'theatre', 'dairy',
'shopping mall', 'grocery', 'supermarket', 'bus station', 'police station', 'railway station', 'bank',
'atm', 'hospital', 'clinic']


# load the labeled dataset from a CSV file

df = pd.read_csv('dataset_honors_exp3_amenity.csv')

df = df.rename(columns={'query':'sentence', 'category':'category'})


# load the Universal Sentence Encoder model

model = hub.load("https://tfhub.dev/google/universal-sentence-encoder/4")


# encode the training data using the USE model

X_train = []

y_train = []

for i, row in df.iterrows():

    sentence = row['sentence']

    category = row['category']

    embeddings = model([sentence])[0].numpy()

    X_train.append(embeddings)

    y_train.append(category)


X_train = np.stack(X_train)
```

```
y_train = np.array(y_train)

# define the input sentence
input_text = "I want to watch a movie"

# encode the input sentence using the USE model
input_embedding = model([input_text])[0].numpy()

# compute cosine similarity between input embedding and training embeddings
sims = cosine_similarity(X_train, input_embedding.reshape(1, -1))

# get the top 5 categories with highest similarity scores
top_indices = np.argsort(sims.ravel())[-5:][::-1]
top_categories = y_train[top_indices]
top_scores = sims[top_indices]

# print the top 5 categories with their similarity scores
for category, score in zip(top_categories, top_scores):
    print(f"{category}: {score[0]:.4f}")

# define the input sentence
input_text = "I want to eat ice cream"

# encode the input sentence using the USE model
input_embedding = model([input_text])[0].numpy()

# compute cosine similarity between input embedding and training embeddings
sims = cosine_similarity(X_train, input_embedding.reshape(1, -1))

# get the top 5 categories with highest similarity scores
```

```
top_indices = np.argsort(sims.ravel())[-5:][::-1]
top_categories = y_train[top_indices]
top_scores = sims[top_indices]

# print the top 5 categories with their similarity scores
for category, score in zip(top_categories, top_scores):
    print(f"{category}: {score[0]:.4f}")

# define the input sentence
input_text = "I want to eat something for dinner"

# encode the input sentence using the USE model
input_embedding = model([input_text])[0].numpy()

# compute cosine similarity between input embedding and training embeddings
sims = cosine_similarity(X_train, input_embedding.reshape(1, -1))

# get the top 5 categories with highest similarity scores
top_indices = np.argsort(sims.ravel())[-5:][::-1]
top_categories = y_train[top_indices]
top_scores = sims[top_indices]

# print the top 5 categories with their similarity scores
for category, score in zip(top_categories, top_scores):
    print(f"{category}: {score[0]:.4f}")

# define the input sentence
input_text = "I am looking to learn in college"

# encode the input sentence using the USE model
```

```
input_embedding = model([input_text])[0].numpy()


# compute cosine similarity between input embedding and training embeddings
sims = cosine_similarity(X_train, input_embedding.reshape(1, -1))


# get the top 5 categories with highest similarity scores
top_indices = np.argsort(sims.ravel())[-5:][::-1]
top_categories = y_train[top_indices]
top_scores = sims[top_indices]


# print the top 5 categories with their similarity scores
for category, score in zip(top_categories, top_scores):
    print(f"{category}: {score[0]:.4f}")
```

2. Code to visually map the locations under respective categories, on the map:

```
import folium
from geopy.geocoders import Nominatim
from geopy.exc import GeocoderTimedOut
import requests
import json


# Specify a user agent string for Nominatim
geolocator = Nominatim(user_agent="my-app")


# Specify the address for which you want to get the latitude and longitude
address = "Dwarkadas J. Sanghvi College of Engineering, Vile Parle West, Mumbai,
Maharashtra 400056, India"


# Use geolocator to get the location information for the address
location = geolocator.geocode(address)


# Get the latitude and longitude
```

```
lat, lng = location.latitude, location.longitude

print(lat, lng)


# Create a map centered on the location

m = folium.Map(location=[lat, lng], zoom_start=14)


# Add a marker for the location

folium.Marker([lat, lng], popup=location.address).add_to(m)

# Set up the search parameters (all emergency services)

# query = ["hospital", "clinic", 'doctors', 'pharmacy'] # For plotting within a 2 km radius

# query = ['fire_station', 'police'] # For plotting police stations and fire stations within a 10 km radius

query = ['college']

radius = 12000  # meters (different for different queries)


url = f"https://overpass-api.de/api/interpreter?data=[out:json];node(around:{radius},{lat},{lng})[amenity~'{'|'.join(query)}'];out;"


# Make the API request

response = requests.get(url)


# Get the JSON data from the response

data = response.json()


# Loop through the results and add markers to the map

for n, result in enumerate(data["elements"]):

    if "lat" in result and "lon" in result:

        # print(n)

        lat, lon = result["lat"], result["lon"]

        location = geolocator.reverse(f"{lat},{lon}")
```

```
    # folium.Marker([lat, lon], icon=folium.Icon(color='red')).add_to(m)

    folium.Marker([lat, lon], popup=location.address,
icon=folium.Icon(color='red')).add_to(m)


# Display the map

m.save("map_custom.html")
```

Output:

The following is the output when different contexts are classified by classifier:

1. “I want to watch a movie”

theatre: 0.8953

theatre: 0.5967

shopping mall: 0.5523

theatre: 0.5443

theatre: 0.5172

2. “I want to eat ice cream”

dairy: 0.7129

ice cream: 0.6917

ice cream: 0.5981

ice cream: 0.5610

dairy: 0.5488

3. “I want to eat something for dinner”

supermarket: 0.6936

restaurant: 0.6917

grocery: 0.6864

restaurant: 0.6747

restaurant: 0.6742

4. “I am looking to learn in college”

cafe: 0.5952

college: 0.5301

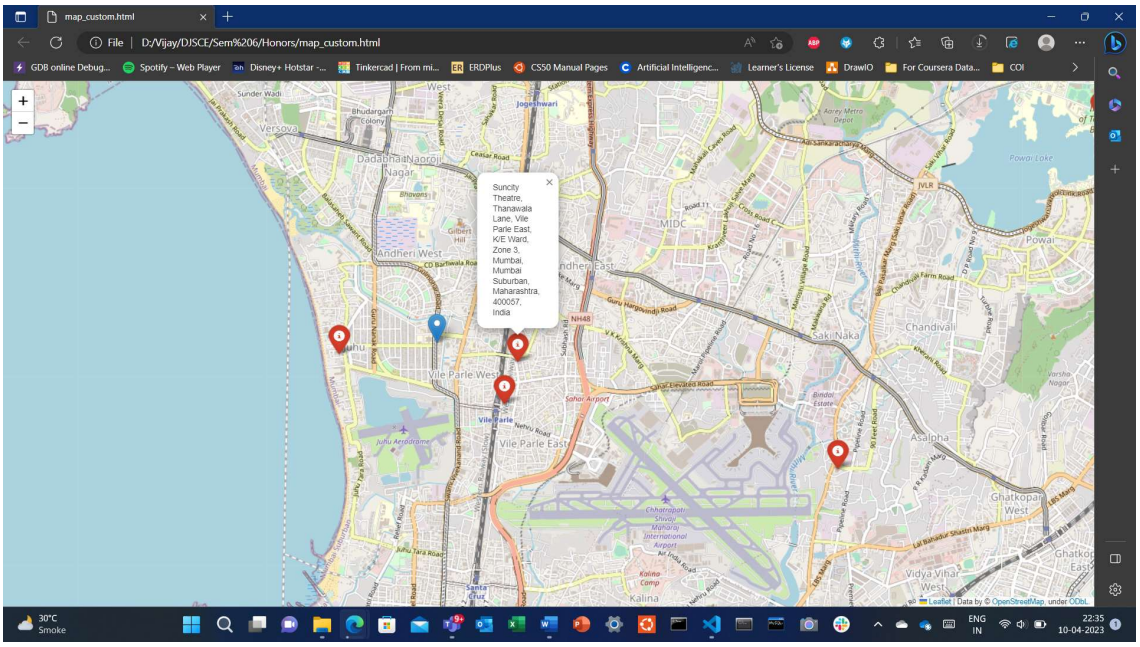
college: 0.5183

college: 0.5109

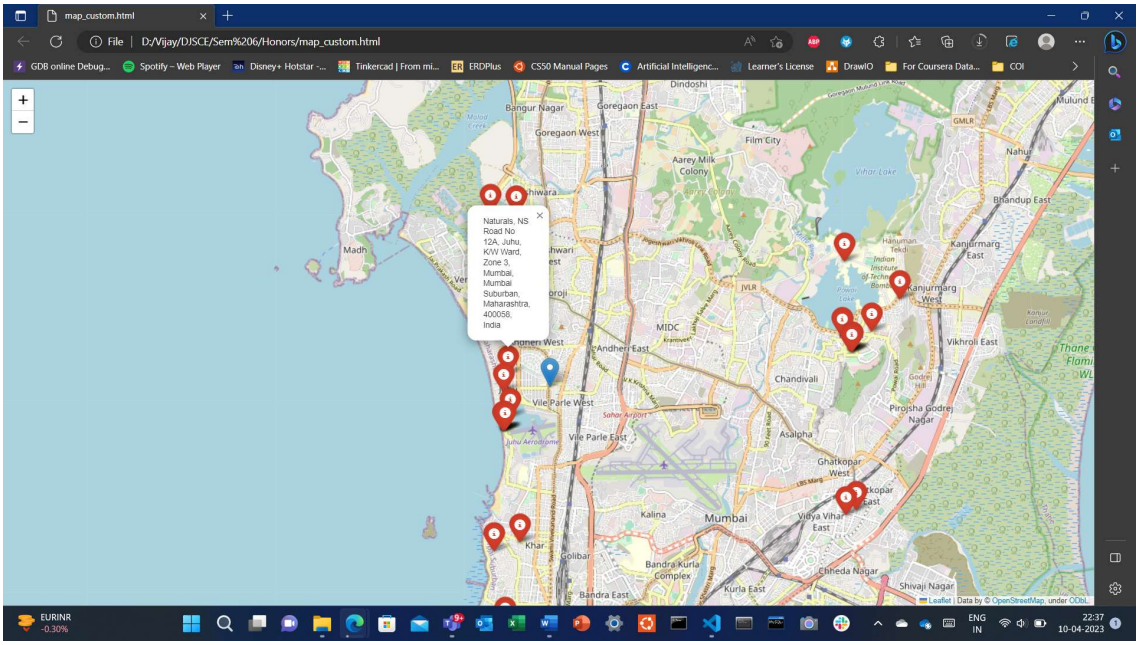
college: 0.5075

After the above categories are returned by the classifier, we plot the places that fall in those categories on the map as follows:

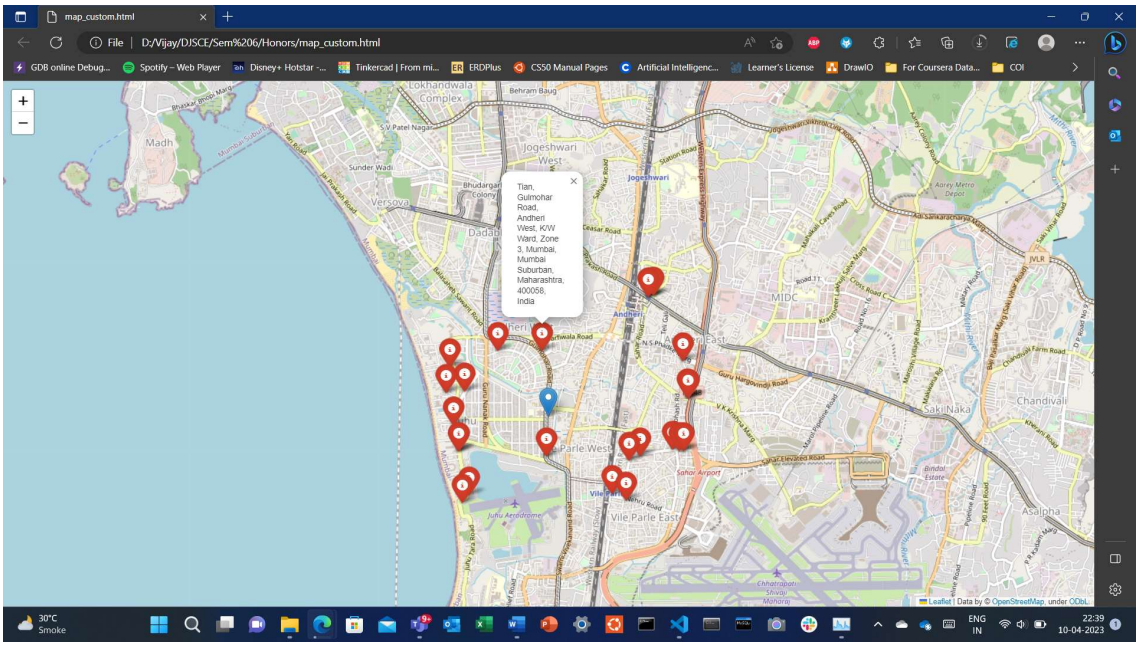
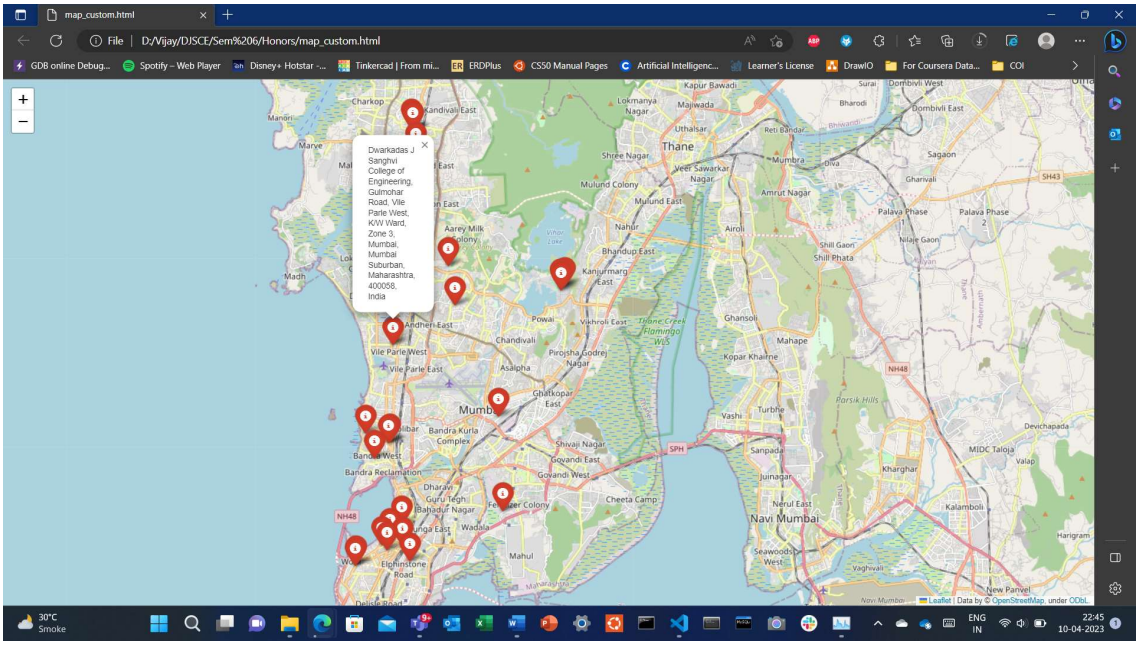
1. Theatre



2. Ice cream parlour



3. Restaurants



4. College